

Rescheduling Production on Identical Parallel Machines upon new jobs arrivals

Nathália C. O. Silva* Cassius T. Scarpin** Angel Ruiz*** José E. Pécora Jr.****

**Instituto Federal do Rio Grande do Sul, Erechim, RS, Brazil, (e-mail: nathalia.silva@erechim.ifrs)*

***Grupo de Tecnologia Aplicada à Otimização, Universidade Federal do Paraná, Curitiba, PR, Brazil, (e-mail: cassiusts@ufpr.br)*

****Centre Interuniversitaire de Recherche sur le Réseaux d'Entreprise, la Logistique et le Transport, Université Laval, Québec, Canada, (e-mail: angel.ruiz@osd.ulaval.ca)*

*****Grupo de Tecnologia Aplicada à Otimização, Universidade Federal do Paraná, Curitiba, PR, Brazil, (e-mail: pecora@ufpr.br)*

Abstract: This paper deals with a dynamic scheduling problem where a production schedule must be adapted to cope with the arrival of new orders. The problem encompasses several identical parallel machines with sequence-dependent setup times and a release date for each job. The problem's goal is to minimize both the makespan and the total tardiness of the jobs. A Mixed Integer Linear Programming model is used to generate the initial schedule and then, every time a new order arrives, the MIP is executed again but considering the new order plus the orders not completed so far executed. The results produced for a tested of 16 different problems are analyzed according to two parameters: (1) the number of new jobs arriving to the system, and (2) the relative weight granted to each of the objectives.

© 2019, IFAC (International Federation of Automatic Control) Hosting by Elsevier Ltd. All rights reserved.

Keywords: Rescheduling; Dynamic System; Identical Parallel Machines; Makespan; Total Tardiness; Exact Approach.

1. INTRODUCTION

Production scheduling is an extremely important problem for decision-makers in the current manufacturing environment. Companies seek not only to minimize production costs but also to guarantee high customer service without affecting the established deadlines (Allahverdi et al., 2014). Among the production environments commonly found in the literature, this study focuses on the identical parallel machines multiobjective scheduling problem, considering sequence-dependent setup times and release dates of jobs. This problem has two objectives: to minimize the maximum completion time of jobs, i.e., the makespan, and to minimize the total tardiness of jobs. Many authors have addressed similar problems in the past, especially for the case of makespan minimization (Behnamian et al., 2011; Jia et al., 2017). In other cases, the goal is rather to minimize costs associated to tardiness of jobs, for instance, when it is wished to minimize deterioration of perishable products or avoid customers dissatisfaction due to delays. According to the three-field notation, the problem is denoted by $Pm|s_{ij}, r_j|C_{max}, \sum T_j$, where m determines the number of identical parallel machines, s_{ij} is the setup time between jobs i and j , and r_j is the release date. For the objectives, C_{max} represents the makespan and $\sum T_j$ determines the total tardiness of jobs.

It is generally assumed that all the information necessary to schedule the jobs is known at the beginning of the production process. However, in a dynamic environment, changes may occur during the schedule execution. This study focuses on such changes by addressing production scheduling in real time

considering that new orders arrive during the production process. Thus, upon a new order arrival, the jobs not yet processed plus the new one need to be “rescheduled”. In this paper, the rescheduling is done by an exact approach that uses a Mixed Integer Linear Programming (MILP) model. The goal of this paper is therefore to empirically assess the impacts of the changes introduced by the successive reschedules with respect to the initial one. The remainder of the paper is organized as follows: Section 2 presents a brief literature review on the addressed topic. Section 3 formulates the Mixed Integer Linear Programming model for identical parallel machine multiobjective scheduling problem. Section 4 presents the production rescheduling method. Computational tests are presented and discussed in Section 5. Final considerations are reported in Section 6.

2. LITERATURE REVIEW

A large number of recent studies address identical parallel machines multiobjective scheduling problems (Baesler and Palma, 2014; Ruiz-Torres et al., 2008; Ruiz-Torres et al., 2018). Other important features characterizing scheduling problems are the setup time (especially, in the dependent-sequence cases), and the release dates of jobs. These characteristics are directly related to the needs in the industrial sector, especially in the textile, chemical, pharmaceutical, food and metallurgical sectors (Behnamian et al., 2011). On the other hand, although the makespan and total tardiness minimization are the main objectives in scheduling problems, few studies address these objectives simultaneously for identical parallel machines scheduling problems. Among them, Behnamian et al. (2010) proposed a multi-phase method

to do it. In the first phase, the authors developed a Based Random-Key Genetic Algorithm (BRKGA) to discover a good approximation to the Pareto-optimal front. In the second and third phases, improvements are made at the Pareto front by a hybrid metaheuristic that combines Ant Colony Optimization (ACO), Variable Neighborhood Search (VNS), and Simulated Annealing (SA). On a subsequent work, Behnamian et al. (2011) proposed a hybrid metaheuristic that encompasses the following steps: first, an ACO creates an initial population, then the SA is used to prevent that the solution becomes a local optimal, and finally, the VNS and a Local Search are applied for solution improvement. Also, the combination of two methods (SA/VNS and ACO/VNS) is proposed to solve the problem. Zarandi and Kayvanfar (2015) also addressed a similar scheduling problem but, contrarily to the previous works, in this case the processing times are controllable, i.e., it is allowed the compression and expansion of their values. A heuristic defines the values of this parameter. Two algorithms, the Non-dominated Genetic Sorting Metaheuristics Algorithm II (NSGA-II) and the Non-dominated Genetic Ranking Algorithm (NRGA), are used to solve this problem.

To the best of our knowledge, Li et al. (2012a) and Li et al. (2012b) are the only papers in the literature that use sequence-dependent setup time and release date of jobs for an identical parallel machines multiobjective scheduling problem, in order to minimize the makespan and the total tardiness of jobs. The first work proposed a Mixed Integer Linear Programming (MILP) model and, due to the complexity of the problem, they applied the NSGA-II metaheuristic to solve it. After this, they linked a fuzzy logic controller to the NSGA-II, creating the Fuzzy Logic Controller with the Non-dominated Sorting Genetic Algorithm II (FLC-NSGA-II). Also, they developed an exact approach based on the two-phase method to identify the Pareto-optimal solutions in small instances. In the second paper, the authors compared the proposed metaheuristics FLC-NSGA-II and FLC-SPEA-II, adapted from the NSGA-II and SPEA-II, respectively. For this, the authors developed a fuzzy logic controller responsible for the improvement of the parameters that perform the crossover and mutation process.

A number of works describing real production applications are reviewed in Allahverdi and Soroush (2008). Among them, Baesler and Palma (2014) addressed the identical parallel machines scheduling problem in a sawmill industry. The authors proposed the Memetic Algorithm with Compromise Search (MACS) metaheuristic that combines genetic evolution and Local Search concepts to solve the problem. To summarize, multiobjective optimization has been increasingly applied in the productive system. However, despite of these efforts, there is still much to be explored in this area. Besides, we can note that few papers use dynamic characteristics in the performed scheduling and, to the best of our knowledge, no study has addressed a production rescheduling due to the insertion of new customers' orders.

3. PROBLEM DESCRIPTION

The multiobjective scheduling problem in this paper is denoted by $Pm|s_{ij}, r_j|\lambda C_{max} + (1 - \lambda)\sum T_j$ and, as we mentioned

before, it has very similar characteristics to the paper of (Li et al., 2012a). Thus, the mathematical formulation is an adaptation of the MILP model proposed by the authors. The following assumptions are considered:

- Each job can be processed by only one machine and, consequently, each machine performs only one job at the same time.
- A dummy job is considered for the system initialization, i.e., the dummy job precedes immediately the first job on each machine.
- Preemption is not allowed during the scheduling, i.e., no job can be interrupted during its execution.

In the identical parallel machines scheduling problem, each job in $N = \{1, 2, \dots, n\}$ must be performed on a machine in $M = \{1, 2, \dots, m\}$. The notations used are described below.

Indexes:

i, j : jobs indexes;

k : machines index.

Input parameters (integers and non-negatives):

r_j : release date of job j ;

d_j : due date of job j ;

p_j : processing time of job j ;

s_{ij} : setup time between jobs i and j (i is immediately predecessor to j).

Decision variables:

C_{jk} : completion instant of job j at the machine k ;

$T_j = \max\{0, C_{jk} - d_j\}$: tardiness of job j ;

$C_{max} = \max C_j$: makespan;

$x_{ijk} = 1$, if the job i is immediately predecessor to j at the machine k , and 0 otherwise.

The objective function and the constraints of the problem are as follows:

$$\min Z = \lambda C_{max} + (1 - \lambda) \sum_{j=1}^n T_j \quad (1)$$

Subject to:

$$\sum_{k=1}^m \sum_{i=0}^n x_{ijk} = 1, \quad \forall j = 1, \dots, n \quad (2)$$

$$\sum_{k=1}^m \sum_{j=1}^n x_{ijk} \leq 1, \quad \forall i = 1, \dots, n \quad (3)$$

$$\sum_{j=1}^n x_{0jk} \leq 1, \quad \forall k = 1, \dots, m \quad (4)$$

$$\sum_{\substack{h=0 \\ h \neq i, j}}^n x_{hik} \geq x_{ijk}, \quad \forall i, j = 1, \dots, n, \quad i \neq j, \quad \forall k = 1, \dots, m \quad (5)$$

$$C_{jk} - C_{ik} \geq s_{ij} + p_j + (x_{ijk} - 1)M, \quad \forall i = 0, \dots, n, \quad \forall j = 1, \dots, n, \quad i \neq j, \quad \forall k = 1, \dots, m \quad (6)$$

$$C_{0k} = 0, \quad \forall k = 1, \dots, m \quad (7)$$

$$C_{jk} \geq r_j + p_j, \quad \forall j = 1, \dots, n, \quad \forall k = 1, \dots, m \quad (8)$$

$$C_{max} \geq C_{jk}, \quad \forall j = 1, \dots, n, \quad \forall k = 1, \dots, m \quad (9)$$

$$C_{jk} \geq 0, \quad \forall j = 1, \dots, n, \quad \forall k = 1, \dots, m \quad (10)$$

$$T_j \geq C_{jk} - d_j, \quad \forall j = 1, \dots, n, \quad \forall k = 1, \dots, m \quad (11)$$

$$T_j \geq 0, \quad \forall j = 1, \dots, n \quad (12)$$

$$x_{ijk} \in \{0,1\}, \quad \forall i = 0, \dots, n, \quad \forall j = 1, \dots, n, \quad i \neq j, \quad \forall k = 1, \dots, m \quad (13)$$

The bi-objective function in (1) is responsible for minimizing the convex combination of the makespan and the total tardiness, simultaneously. The parameter λ ($0 \leq \lambda \leq 1$) denotes the weight or priority given to each of the objectives in the function.

The constraints set (2) ensures that each job, associated with only one machine, has just one job immediately predecessor. Constraints (3) guarantee that there is at most one job immediately succeeding the other, if both jobs are performed by the same machine. Constraints (4) inform that the dummy job 0, responsible for the system initialization, can be succeeded by only one job at each machine. Constraints (5) ensure that the predecessors and successors among themselves are performed by the same machine. Constraints (6) certify that the completion time of job j must occur after the sum of the completion time of job i (immediately predecessor to j), the setup time between jobs i and j , and the processing time of job j . Equations (7) establish that the completion time of the dummy job is null for all the machines. Constraints (8) determine that the job is performed after its release date. In (9), the maximum completion time of jobs must be greater than or equal to the completion time of each job for all machines. Constraints (10) enforce jobs' completion times to be non-negative. The constraints sets (11) and (12) are responsible for the tardiness value related to job j . In (11), the lateness is calculated by the difference between the completion time of job j and its due date. In turn, in constraints (12), only the positive values of the lateness are considered. Finally, constraints (13) define the domain for decision variables x_{ijk} .

4. SIMULATED REAL-TIME DECISION PROCESS

In most theoretical production systems, all the jobs are available at the beginning of the planning horizon or, less frequently, their release dates are known a priori. On the other hand, many changes happen during production in real-life environments. For these cases, it is required to modify or adapt

the original schedule whenever a change happen in such a way that the impacts on the previous service engagements are minimized. In this paper we consider a realistic production system in which new orders can be added to the production schedule during the execution of the jobs initially considered. To tackle this dynamic problem, we propose a real-time decision process which, each time that a new order arrives to the system, triggers the rescheduling of the jobs that have not been completed so far plus the arrived order. The next paragraphs describe this process in details and define how the events are generated and characterized.

Let us assume that, at an arbitrary initial time, an initial set of jobs is given. An initial schedule (S_0) is generated for this set of jobs using the formulation already presented. Based on this schedule and the associated makespan, a number of random events (i.e. arrival of new jobs) will be created to reproduce to real behavior of industrial settings where jobs arrive in an unpredictable manner. The first parameter to set is R , the number of events to generate. We elected to make vary R within the interval $(0.2 N; 0.5 N)$. For example, if the system has 10 initial jobs, we consider the insertion of 2, 3, and up to 5 jobs in the schedule during the planning horizon. The arrival time of the first new job is drawn from a uniform distribution between the initial time and the makespan of the initial sequence, i.e.,

$$t(E_1) \in (0, C_{\max(0)}]; \quad (14)$$

where E_1 denotes the first event and $C_{\max(0)}$ is the initial makespan value. The arrival times of the subsequent events are generated by adding to the arrival time of the previous event a random time drawn from a uniform distribution as follows:

$$t(E_i) \in (t(E_{i-1}), C_{\max(0)}]; \quad i \geq 2. \quad (15)$$

The Rescheduling Algorithm presents the steps performed in a production system with the insertion of new orders during the execution of the original scheduling, as previously described.

Rescheduling Algorithm

1. Generate de initial schedule S_0
 2. **For** $i = 1$ **to** R
 3. Start the execution of the listed jobs in the schedule S_{i-1} until the arrival time of event i
 4. List the jobs not yet processed in the schedule S_{i-1} , and the new event in a new schedule S_i
 - End for**
 5. Execute the jobs in the schedule S_i
 6. **End**
-

Based on the initial schedule, we start the execution of the jobs until the arrival time of the first event. If some machine is processing a job at this moment, first we finalize the execution of this job, and later we start the rescheduling process. At this point, all the jobs not yet initialized at initial schedule plus the new event are processed in a new schedule, named S_1 . This rescheduling process starts at the end of the last job executed on each machine when the event was generated. The Mixed Integer Linear Programming model, mentioned in Section 3, is used to produce a solution to this rescheduling problem.

Hence, we preserve the sequence of jobs already performed and we reorganize the ones that are still in the waiting queue according to their specific priority. If a new event be introduced, another schedule is generated (named schedule S_2) and the previous step is repeated. The process finishes when all the events have been scheduled.

It is worth mentioning the fact that the jobs' sequence can be modified at each rescheduling. Thus, the final schedule (after the insertion of all events) is composed of several partial schedules elaborated as information (the arrival of events) is revealed.

5. COMPUTATIONAL RESULTS

The rescheduling method was implemented in C++ and the mathematical formulation was solved by Gurobi 8.0.0, with default parameters. The tests were executed on a computer with operating system Microsoft Windows 10 (64 bit), with AMD A10 PRO-7800 B R7 processor, 12 Compute Cores 4C+8G 3.50 GHz and 8 GB RAM.

Since this problem is well-known to be NP-Hard, the rescheduling method by an exact approach cannot be applied to large instances. Thus, we generate a tested a set of small sized instances based on the ones proposed in (Li et al., 2012a), for $N = 10$ jobs and $M = 3$ identical machines, to consider:

- Processing time of job j (p_j): it is generated by a uniform distribution in the interval $[1, 100]$.
- Setup time between jobs i and j (s_{ij}): in the first case, we consider a uniform distribution of 10% to 20% of the jobs processing time, and in the second case, a we consider a uniform distribution of 10% to 50% of the jobs processing time. That is, we determine uniformly the setup time in the intervals $[0.1p_j, 0.2p_j]$ and $[0.1p_j, 0.5p_j]$.
- Release date of job j (r_j): $[0, 50N \frac{\beta}{M}]$; where N represents the quantity of jobs, M is the number of available machines and β is a parameter with the values 0.6 and 0.8.
- Due date of job j (d_j): $[\max(0, P(1 - TF - \frac{RDD}{2})), P(1 - TF + \frac{RDD}{2})]$; where $P = \frac{\sum_{j=1}^n p_j}{M}$ and the parameters TF and RDD are 0.4 and 0.2, respectively.
- Number of insertions of new events: this number is directly related to the number of jobs present in the initial scheduling, so we consider insertions of 20% to 50% of the original quantity of jobs. In this case, we have 2, 3, 4, and 5 insertions in the scheduling.

According to these parameters, 16 different groups of problems referred to as G_1 to G_{16} were created. Table 1 indicates the number of the problem and its configuration.

In addition, we set the parameter λ to the values $\{0, 0.25, 0.5, 0.75, 1\}$ to evaluate the sensibility of the problem with respect to this parameter. A total of 100 replications was performed in each set of problems.

Table 1. Indices of the problems

$s_{ij}; \beta$	Number of Insertions			
	2	3	4	5
$[0.1p_j, 0.2p_j]; 0.6$	G_1	G_2	G_3	G_4
$[0.1p_j, 0.2p_j]; 0.8$	G_5	G_6	G_7	G_8
$[0.1p_j, 0.5p_j]; 0.6$	G_9	G_{10}	G_{11}	G_{12}
$[0.1p_j, 0.5p_j]; 0.8$	G_{13}	G_{14}	G_{15}	G_{16}

5.1 Analysis and Discussion

Tables 2,3,4 and 5, report the results produced for each value of the weight parameter λ . For each scenario, we evaluated the average and standard deviation of the makespan and total tardiness of jobs after the production rescheduling, with a significance level of 95 percent. Table 2 shows the results to $\lambda = 0$, i.e., the scenario where we aim to minimize only the total tardiness of jobs. In this case, the $G_1 - G_4$ problems presented the least impact on the original makespan value, increasing approximately 10% (for 2 insertions) up to 29% (for 5 insertions). In relation to the total tardiness of jobs, we can observe that the $G_1 - G_4$ set also resulted in the smaller increase in its original values (increasing approximately 32% up to 35% in the total tardiness), while the $G_5 - G_8$ and $G_9 - G_{12}$ sets doubled their initial values for the case where 5 insertions were performed.

Table 2. Makespan and total tardiness in the rescheduling to $\lambda = 0$

Initial Values	$\lambda = 0$			
	C_{max}		$\sum T_j$	
	Average	SD	Average	SD
$G_1(C_{max}, \sum T_j)$				
$G_1(238, 366)$	262.86	24.87	482.73	80.00
$G_2(238, 366)$	278.18	31.83	488.57	70.59
$G_3(238, 366)$	292.09	29.28	496.57	82.31
$G_4(238, 366)$	308.03	28.84	494.67	81.03
$G_5(202, 175)$	241.51	27.45	309.55	81.01
$G_6(202, 175)$	260.52	24.80	330.77	85.98
$G_7(202, 175)$	281.57	26.00	362.79	102.62
$G_8(202, 175)$	298.16	29.60	366.25	98.19
$G_9(243, 193)$	278.77	30.26	328.78	80.23
$G_{10}(243, 193)$	310.95	28.20	364.68	102.39
$G_{11}(243, 193)$	327.79	31.51	389.68	97.15
$G_{12}(243, 193)$	343.13	35.65	385.68	100.78
$G_{13}(191, 282)$	230.03	20.46	401.72	60.53
$G_{14}(191, 282)$	246.98	20.24	441.66	80.49
$G_{15}(191, 282)$	265.20	21.51	445.42	86.86
$G_{16}(191, 282)$	278.16	25.81	454.92	95.70

Table 3 reports the results for $\lambda = 0.25$. In this case, the $G_5 - G_8$ set has the smallest increase in the average of the makespan in relation to its initial value, increasing approximately 19% to

2 insertions up to 46% to 5 new jobs. All the other sets of this scenario presented similar results to the makespan in the production rescheduling. For the total tardiness values, we verified that this scenario obtained very close values to those in the previous case. This fact occurs due to the small change in weight parameter (from 0 to 0.25). Thus, the $G_1 - G_4$ set achieved, on average, the smallest increases in its original values (approximately 33% up to 35% of increasing).

Table 3. Makespan and total tardiness in the rescheduling to $\lambda = 0.25$

Initial Values	$\lambda = 0.25$			
	C_{max}		ΣT_j	
$G_i(C_{max}, \Sigma T_j)$	Average	SD	Average	SD
$G_1(205, 367)$	258.00	31.15	488.78	80.30
$G_2(205, 367)$	275.45	35.16	495.61	74.22
$G_3(205, 367)$	290.54	30.07	501.90	83.21
$G_4(205, 367)$	305.28	29.04	497.26	85.84
$G_5(202, 175)$	241.22	28.54	310.68	81.50
$G_6(202, 175)$	259.98	24.80	330.83	85.96
$G_7(202, 175)$	279.01	24.98	365.86	105.80
$G_8(202, 175)$	294.73	27.66	368.23	100.25
$G_9(211, 201)$	279.93	31.31	348.83	74.51
$G_{10}(211, 201)$	309.12	25.46	378.81	96.25
$G_{11}(211, 201)$	326.68	31.26	403.89	92.69
$G_{12}(211, 201)$	342.14	36.16	403.10	99.18
$G_{13}(182, 282)$	228.87	22.61	408.27	62.12
$G_{14}(182, 282)$	244.37	19.01	446.37	82.84
$G_{15}(182, 282)$	260.64	20.35	449.23	89.68
$G_{16}(182, 282)$	272.50	23.64	460.28	97.49

Table 4 reports the results for $\lambda = 0.5$, i.e., both objectives have the same importance. To the makespan values, the $G_{13} - G_{16}$ set obtained the best results for this scenario, with approximately 25% of increasing for 2 insertions up to 49% for 5 insertions. The rest of the problems in this case obtained similar results to the $G_{13} - G_{16}$ set. While the total tardiness values were smaller for the $G_1 - G_4$ set, as in the previous cases. These problems increased their values in approximate 33% up to 36% in relation to the original ones. The other sets of problems obtained a very high increase, when compared to the original scheduling.

Table 5 reports the results for $\lambda = 0.75$. As in the previous case, the best values for the makespan and the total tardiness are produced for the instances sets $G_{13} - G_{16}$ and $G_1 - G_4$, respectively. We can note that, for the intermediate values of λ ($\lambda = 0.25$, $\lambda = 0.5$ and $\lambda = 0.75$), the increase in both objectives function values was very similar for all the problems addressed.

Table 4. Makespan and total tardiness in the rescheduling to $\lambda = 0.5$

Initial Values	$\lambda = 0.5$			
	C_{max}		ΣT_j	
$G_i(C_{max}, \Sigma T_j)$	Average	SD	Average	SD
$G_1(195, 372)$	257.84	33.28	495.73	76.08
$G_2(195, 372)$	276.27	34.78	501.49	69.92
$G_3(195, 372)$	291.32	30.17	506.50	77.31
$G_4(195, 372)$	305.13	28.61	505.30	78.04
$G_5(190, 185)$	243.16	27.73	323.07	75.59
$G_6(190, 185)$	262.42	22.82	344.15	79.55
$G_7(190, 185)$	281.49	27.58	374.85	102.55
$G_8(190, 185)$	297.07	29.33	378.64	98.51
$G_9(211, 201)$	278.44	31.18	352.50	76.43
$G_{10}(211, 201)$	309.25	27.02	383.47	99.31
$G_{11}(211, 201)$	325.30	30.09	409.48	94.48
$G_{12}(211, 201)$	339.44	35.32	407.93	100.41
$G_{13}(182, 282)$	227.93	22.45	409.67	62.89
$G_{14}(182, 282)$	244.24	19.07	448.38	83.86
$G_{15}(182, 282)$	258.58	20.78	450.56	90.44
$G_{16}(182, 282)$	271.31	22.11	462.04	98.52

Table 5. Makespan and total tardiness in the rescheduling to $\lambda = 0.75$

Initial Values	$\lambda = 0.75$			
	C_{max}		ΣT_j	
$G_i(C_{max}, \Sigma T_j)$	Average	SD	Average	SD
$G_1(195, 372)$	257.50	33.55	498.11	78.65
$G_2(195, 372)$	275.18	35.04	506.44	72.52
$G_3(195, 372)$	290.91	30.77	511.73	81.62
$G_4(195, 372)$	304.78	28.38	507.83	81.62
$G_5(189, 188)$	243.63	28.48	342.82	86.58
$G_6(189, 188)$	262.34	22.88	357.33	87.98
$G_7(189, 188)$	279.82	26.20	381.65	106.57
$G_8(189, 188)$	295.61	29.97	385.81	104.52
$G_9(211, 201)$	277.68	32.23	356.23	78.09
$G_{10}(211, 201)$	308.84	27.52	388.06	102.45
$G_{11}(211, 201)$	326.41	32.02	414.94	98.79
$G_{12}(211, 201)$	339.65	32.92	417.47	103.54
$G_{13}(178, 294)$	227.70	22.87	420.46	60.63
$G_{14}(178, 294)$	243.68	20.76	461.52	81.32
$G_{15}(178, 294)$	258.21	19.56	462.49	87.29
$G_{16}(178, 294)$	269.65	22.13	474.61	95.91

The last scenario considers $\lambda = 1$, which consists in minimizing the makespan. In this case, the $G_5 - G_8$ set resulted in the smaller increase in the makespan values, approximately 26% (for 2 insertions) up to 56% (for 5 insertions). The makespan values in the other sets had no

significant difference from this one. For the total tardiness case, the best result was obtained for the $G_1 - G_4$ set, with approximately 39% up to 46% of increasing in the total tardiness of jobs. However, the other sets had a great difference in relation to the increase in the total tardiness after the rescheduling process, with approximately 124% of increasing for 5 insertion of new orders in the $G_5 - G_8$ set.

Table 6. Makespan and total tardiness in the rescheduling to $\lambda = 1$

Initial Values	$\lambda = 1$			
	C_{max}		$\sum T_j$	
$G_i(C_{max}, \sum T_j)$	Average	SD	Average	SD
$G_1(193, 474)$	258.01	32.47	656.80	128.23
$G_2(193, 474)$	276.04	34.61	670.10	130.51
$G_3(193, 474)$	291.19	29.37	684.58	153.14
$G_4(193, 474)$	305.61	28.01	690.08	148.88
$G_5(187, 237)$	235.04	30.16	420.22	105.24
$G_6(187, 237)$	256.12	27.16	441.95	114.34
$G_7(187, 237)$	278.62	28.12	526.11	149.33
$G_8(187, 237)$	292.03	26.94	530.88	160.79
$G_9(211, 240)$	274.76	30.70	427.61	111.92
$G_{10}(211, 240)$	306.96	27.18	472.54	159.93
$G_{11}(211, 240)$	325.07	31.13	522.06	143.64
$G_{12}(211, 240)$	337.52	34.83	524.54	155.18
$G_{13}(174, 352)$	227.53	24.31	506.28	88.32
$G_{14}(174, 352)$	243.99	20.13	557.12	115.99
$G_{15}(174, 352)$	258.71	20.06	579.08	118.94
$G_{16}(174, 352)$	269.90	22.02	578.81	128.53

For the makespan minimization, the 16 problems resulted in similar increases for all parameters tested. However, we observed a significant difference for the total tardiness cases. For all parameters evaluated, the $G_1 - G_4$ set achieved the best results (approximately 32% up to 46%), while the $G_5 - G_8$ and $G_9 - G_{12}$ sets resulted in significant increases when compared to the initial total tardiness values (more than 100% of increasing, in some cases).

6. CONCLUSIONS

In this paper we studied the dynamic production scheduling process where new orders or jobs arrive during the planning horizon. A mixed integer formulation is used to reschedule uncompleted jobs upon the arrival of each new order with the objective of simultaneously minimize the makespan and total tardiness. To get some insight on how both the makespan and the tardiness are sensible to the number of new jobs and the preferences of decision-makers, numerical experiments were conducted on a small-sized tesbed. We evaluated 16 different problems, each containing 10 jobs to be processed in 3 identical parallel machines. The number of new jobs varied from 20% to 50% of the initial number of jobs, and the parameter λ , which defines the relative importance of the two objectives, was set to $\{0, 0.25, 0.5, 0.75, 1\}$. We observed that,

in average, the increase in the makespan after the rescheduling process had smaller variations when compared to the original values. We noted that the largest increase is slightly more than 50% of the its original value. On the other hand, the increase of the total tardiness of jobs obtained great changes in their values for some evaluated problems. The $G_1 - G_4$ set guaranteed the lowest increases for all weight parameters in the objective function (32% to 46% in relation to the initial scheduling). However, some problems resulted in values higher than the 100% of the original scheduling values, especially in the $G_5 - G_8$ and $G - G_{12}$ sets.

In future research, we intend to apply heuristics or metaheuristics methods to this process in order to be able to solve efficiently larger instances.

REFERENCES

- Allahverdi, A., Aydilek, H., Aydilek, A. (2014). Single machine scheduling problem with interval processing times to minimize mean weighted completion time, *Comput. Oper. Res.*, 51, 200–207.
- Allahverdi, A., Soroush, H.M. (2008). The Significance of Reducing Setup Times/Setup Costs, *Eur. J. Oper. Res.*, 187, 978–984.
- Baessler, F., Palma, C. (2014). Multiobjective parallel machine scheduling in the sawmill industry using memetic algorithms, *Int. J. Adv. Manuf. Technol.*, 74, 757–768.
- Behnamian, J., Zandieh, M., Fatemi Ghomi, S.M.T. (2010). A multi-phase covering Pareto-optimal front method to multi-objective parallel machine scheduling, *Int. J. Prod. Res.*, 48, 4949–4976.
- Behnamian M; Zandieh, J., Ghomi, S.M.T.F. (2011). Bi-objective parallel machines scheduling with sequence-dependent setup times using hybrid metaheuristics and weighted min-max technique, *Soft Comput*, 15, 1313–1331.
- Jia, Z., Pei, M., Leung, J.Y.-T. (2017). Multi-objective ACO algorithms to minimise the makespan and the total rejection cost on BPMs with arbitrary job weights, *Int. J. Syst. Sci.*, 48, 3542–3557.
- Li, X., Yalaoui, F., Amodeo, L., Chehade, H. (2012). Metaheuristics and exact methods to solve a multiobjective parallel machines scheduling problem, *J. Intell. Manuf.*, 23, 1179–1194.
- Li, X; Chehade, H; Yalaoui, F., Amodeo, L. (2012). Fuzzy logic controller based multiobjective metaheuristics to solve a parallel machines scheduling problem, *J. Mult.-Valued Log. Soft Comput.*, 18, 617–636.
- Ruiz-Torres, A.J., Lpez, F.J., Ho, J.C., Wojciechowski, P.J. (2008). Minimizing the average tardiness: the case of outsource machines, *Int. J. Prod. Res.*, 46, 3615–3640.
- Ruiz-Torres, A.J., Paletta, G., Mahmoodi, F., Ablanedo-Rosas, J.H. (2018). Scheduling assemble-to-order systems with multiple cells to minimize costs and tardy deliveries, *Comput. Ind. Eng.*, 115, 290–303.
- Zarandi, M.H.F., Kayvanfar, V. (2015). A bi-objective identical parallel machine scheduling problem with controllable processing times: a just-in-time approach, *Int. J. Adv. Manuf. Technol.*, 77, 545–563.